

Module	Week	Session	Title	Topics Covered	Learning Objectives	To be taken by
Module 1: Foundations of Data	1	1.1	The AI Landscape & Programming Foundations	AI vs ML vs GenAI; industry use cases; VS Code & Colab setup; Git/GitHub basics; API keys & secrets	Distinguish AI, ML and GenAI using real-world examples; set up a working dev environment (VS Code, Colab, Git/GitHub); securely manage API keys in a project	Industry Mentor
Module 1: Foundations of Data	1	1.2	Python Fundamentals	Variables, data types (int, float, str, bool), operators, input/output, f-strings, notebook discipline in Colab	Identify and declare variables using correct data types; construct expressions using arithmetic and comparison operators; write and execute programs using input/output and f-strings in Colab	Industry Mentor
Module 1: Foundations of Data	2	2.1	Control Flow & Decision Making	if, elif, else, boolean logic, comparison operators, nested conditions	Construct if/elif/else blocks to handle real decision scenarios; evaluate boolean expressions combining multiple conditions; trace execution through nested conditions and predict output	Industry Mentor
Module 1: Foundations of Data	2	2.2	Loops, Iteration & Repetitive Logic	for loops, while loops, range(), break, continue, iterating over lists and strings	Write for and while loops with correct termination conditions; apply break and continue to control loop flow; iterate over sequences and access elements by index	Industry Mentor
Module 1: Foundations of Data	3	3.1	Master class: Numbers, Logic & Structure — The Mathematical Language of Data	Number Systems & Boolean Logic : How computers represent truth. Binary numbers, logical operators (AND, OR, NOT), truth tables, De Morgan's laws. Why this underlies every conditional and every data type. Set Theory & Mathematical Structures : What is a set, subset, union, intersection, complement. How lists, dictionaries, and JSON are just applied set-theoretic structures. Venn diagrams. Mapping and functions in the mathematical sense (domain, codomain, range).	Understand why 0s and 1s can represent any decision a program makes Read and construct truth tables for compound logical expressions See Python's data structures (lists, dicts, sets) as implementations of mathematical sets and functions Understand what a "function" means mathematically before it means anything in code	Professor
Module 1: Foundations of Data	3	3.2	Writing Reusable Code with Functions	def, parameters, arguments, return values, scope, default arguments, code modularity	Define and call functions with parameters and return values; explain how scope affects variable access; refactor repeated code blocks into reusable modular functions	Industry Mentor
Module 1: Foundations of Data	3	3.3	Python Data Structures	Lists, dictionaries, tuples, sets, indexing, slicing, nesting, when to use each	Create and perform operations on lists, dictionaries, tuples and sets; differentiate between mutable and immutable structures; select the appropriate data structure for a given problem	Industry Mentor
Module 1: Foundations of Data	4	4.1	File Handling, JSON & APIs	File I/O, open(), context managers, JSON structure, json. loads/dumps, requests library, GET/POST, status codes, API keys, rate limiting, ethics, ToS	Read and write files safely using context managers; parse and generate JSON data; make API calls using requests, handle responses and apply ethical usage of API keys and rate limits	Industry Mentor
Module 1: Foundations of Data	4	4.2	NumPy: Numerical Foundation	Arrays, dtype, shape, indexing, slicing, element-wise operations, broadcasting, reshape, flatten, multi-dimensional arrays	Create and manipulate NumPy arrays using indexing and slicing; perform element-wise arithmetic and broadcasting operations without explicit loops; reshape and flatten arrays for downstream use	Industry Mentor
Module 1: Foundations of Data	5	5.1	Pandas: Loading, Inspection & Filtering	pd.read_csv(), Series, DataFrame, head(), info(), describe(), shape, boolean indexing, loc, iloc, sorting, column selection	Load a CSV into a DataFrame and inspect its structure using head(), info() and describe(); filter rows using boolean conditions and loc/iloc; identify data quality issues from inspection output	Industry Mentor
Module 1: Foundations of Data	5	5.2	Pandas: Aggregation, Groupby & Merging	groupby(), agg(), value_counts(), missing values, fillna(), dropna(), merge(), join(), concat(), drop_duplicates()	Group and aggregate data to answer business questions using groupby and agg; handle missing values by dropping or filling appropriately; merge and concatenate multiple DataFrames correctly	Industry Mentor
Module 1: Foundations of Data	6	6.1	Master class: From Tables to Relationships - The Mathematics of Data Organisation	Coordinate Geometry & Visualisation Intuition : The Cartesian plane. What it means to "plot" data. Why a scatter plot is just mapping two variables to (x, y) coordinates. Slope of a line rise over run, as the seed of everything in ML. Basic Descriptive Statistics : Mean, median, mode, range, variance, standard deviation. What each tells you and what it hides. Why the mean lies when data is skewed. Real examples from datasets.	Understand why data visualisation is rooted in coordinate geometry Compute and interpret mean, variance, and standard deviation manually and explain what they reveal	Professor
Module 1: Foundations of Data	6	6.2	Data Visualization	Matplotlib (line, bar, scatter, histogram), Plotly (interactive charts), labels, titles, legends, choosing the right chart type	Create line, bar, scatter and histogram plots using Matplotlib; build basic interactive charts using Plotly; select the appropriate chart type based on the variable type and analytical goal	Industry Mentor
Module 1: Foundations of Data	6	6.3	EDA & Business Thinking	EDA checklist, distributions, outliers, skewness, correlation, funnel analysis, conversion rate, retention, cohort thinking, performance metrics	Execute a structured EDA on a raw dataset identifying distributions, outliers and correlations; interpret patterns and anomalies in data; connect data findings to business questions like funnel drop-off and retention	Industry Mentor

Module 1: Foundations of Data	7	7.1	SQL with MySQL Workbench	MySQL Workbench setup, SELECT, WHERE, ORDER BY, LIMIT, DISTINCT, GROUP BY, HAVING, INNER JOIN, LEFT JOIN, RIGHT JOIN, aliases	Write SQL queries to filter, sort and group data in MySQL Workbench; join two or more tables using INNER, LEFT and RIGHT joins; choose the correct join type for a given multi-table scenario	Industry Mentor
Module 1: Foundations of Data	7	7.2	Data Analysis with Spreadsheets	VLOOKUP, XLOOKUP, pivot tables, filters, sorting, conditional formatting, basic formulas (SUM, AVERAGE, COUNTIF), named ranges	Use VLOOKUP and XLOOKUP to retrieve values across sheets; build pivot tables to summarize and slice data; apply COUNTIF and conditional formatting to surface quick data insights	Industry Mentor
Module 2: Classical ML	8	8.1	ML Workflow & Problem Framing	Supervised vs unsupervised, regression vs classification, train/val/test split with sklearn, baselines, cross-validation (cross_val_score), metric selection	Distinguish supervised and unsupervised learning and frame a business problem as the correct ML problem type; split data using train_test_split and evaluate using cross_val_score; justify metric selection based on the problem type	Industry Mentor
Module 2: Classical ML	8	8.2	Data Preparation for ML	SimpleImputer, LabelEncoder, OneHotEncoder, StandardScaler, MinMaxScaler, Pipeline, ColumnTransformer, feature engineering, leakage prevention, SMOTE	Build an end-to-end preprocessing pipeline using sklearn Pipeline and ColumnTransformer; apply correct encoding and scaling strategies for different feature types; identify and prevent data leakage within a preprocessing workflow	Industry Mentor
Module 2: Classical ML	9	9.1	Master Class: The Mathematics Behind Learning: Lines, Curves & Errors	Linear Equations & The Equation of a Line: $y = mx + c$. What the slope and intercept represent in real data. How Linear Regression is literally finding the best line through points. Residuals - the gap between the line and the point - as "error." Calculus Intuition , Rate of Change & Minimisation : What a derivative means conceptually (how fast is something changing?). Why do we want to minimise errors? The idea of a minimum of a curve — where the slope becomes zero. Gradient descent as "walking downhill" no formulas required, only the picture.	1.Derive and interpret the equation of a line from two points 2.Understand what a residual is and why minimising the sum of squared residuals gives you the best fit line 3.Explain what a derivative represents geometrically without computation 4.Describe gradient descent using only the analogy of a hill and a blindfolded person 5.Connect the math of a line to how a machine "learns" from data	Professor
Module 2: Classical ML	9	9.2	Linear Regression	LinearRegression(), fit(), predict(), coefficients, intercept, MAE, RMSE, R^2 using sklearn.metrics, residual analysis, overfitting vs underfitting	Train a LinearRegression model using scikit-learn and generate predictions; evaluate model performance using MAE, RMSE and R^2 ; interpret coefficients and diagnose overfitting by comparing train vs test performance	Industry Mentor
Module 2: Classical ML	9	9.3	Regularization	Ridge, Lasso, ElasticNet from sklearn.linear_model, alpha tuning, bias-variance tradeoff intuition, comparing regularized vs non-regularized models	Apply Ridge and Lasso regression to reduce overfitting and compare with unregularized baseline; tune the alpha hyperparameter and observe its effect on coefficients; explain the bias-variance tradeoff in plain language	Industry Mentor
Module 2: Classical ML	10	10.1	Logistic Regression	LogisticRegression() from sklearn, probability outputs, predict_proba(), sigmoid intuition, binary and multiclass classification, threshold adjustment	Train a LogisticRegression model and interpret predicted probabilities using predict_proba(); adjust the classification threshold and explain its effect on predictions; distinguish binary and multiclass classification setups	Industry Mentor
Module 2: Classical ML	10	10.2	Classification Metrics	confusion_matrix, accuracy_score, precision_score, recall_score, f1_score, roc_curve, roc_auc_score from sklearn.metrics, threshold tuning, precision-recall tradeoff	Compute and interpret all classification metrics using sklearn.metrics; analyze the confusion matrix to explain false positives and false negatives; evaluate the precision-recall tradeoff and select the right metric for a given business scenario	Industry Mentor
Module 2: Classical ML	11	11.1	Master Class Probability & Counting :The Mathematics of Uncertainty	Probability Fundamentals : Sample space, events, $P(A)$, complement rule, addition rule $P(A \cup B)$, multiplication rule for independent events. Classical probability with coins, dice, cards. Conditional probability $P(A B)$ how knowing one thing changes the probability of another. Bayes' Theorem from First Principles : Derived step by step from conditional probability. One concrete real-world example worked out by hand (disease test or spam). The idea that prior belief + new evidence = updated belief. Stated as a formula, not a model.	Define sample space, event, and probability as a ratio Compute $P(A \cup B)$ and $P(A B)$ manually from a small table or Venn diagram Derive Bayes' Theorem from the definition of conditional probability — no shortcuts Work through a complete Bayes' example numerically (e.g., if 1% of population has disease, test is 90% accurate — what is $P(\text{disease} \text{positive test})$?) Explain in plain language why conditional probability is different from joint probability	Professor
Module 2: Classical ML	11	11.2	Decision Trees	DecisionTreeClassifier, max_depth, min_samples_split, Gini impurity, tree visualization with plot_tree, overfitting by depth, feature importance	Train a DecisionTreeClassifier and visualize it using plot_tree; analyze overfitting by comparing train vs test accuracy across depth values; explain a prediction path from root to leaf in plain language	Industry Mentor
Module 2: Classical ML	11	11.3	Random Forests & Ensemble Methods	RandomForestClassifier, n_estimators, max_depth, feature_importances_, bagging intuition, comparison with single decision tree, joblib for model saving	Train a RandomForestClassifier and extract feature importances to explain model behavior; compare random forest vs single decision tree on performance and interpretability; save and load a trained model using joblib	Industry Mentor

Module 2: Classical ML	12	12.1	Model Validation & Leakage	KFold, StratifiedKFold, cross_val_score, GridSearchCV, RandomizedSearchCV, data leakage detection, resampling with SMOTE from imblearn	Implement k-fold and stratified k-fold cross-validation to produce reliable performance estimates; tune hyperparameters using GridSearchCV and interpret results; detect data leakage scenarios and design a leakage-free evaluation pipeline	Industry Mentor
Module 2: Classical ML	12	12.2	Clustering, Model Selection & Explainability	KMeans, elbow method, silhouette_score, model comparison tables, feature importance plots, explaining predictions to non-technical audience	Run KMeans clustering and determine optimal k using the elbow method and silhouette score; compare multiple trained models using a structured metric table; explain model predictions and feature importance in plain business language	Industry Mentor
Module 3: GenAI & Agents	13	13.1	LLM Fundamentals: How Language Models Work	LLM architecture intuition, tokenization, context windows, temperature, top-p, cost and latency basics, LLM vs traditional ML	Recall and explain how LLMs generate text using tokens and context windows; interpret the effect of temperature and top-p on output variability; compare LLMs with traditional ML models in terms of how they learn and predict	Industry Mentor
Module 3: GenAI & Agents	13	13.2	Calling LLMs with APIs	Picking a provider (OpenAI, Anthropic, Gemini, etc.); API keys with python-dotenv; raw calls via requests; message structure (system/user/assistant); parsing responses; error & rate-limit handling	Set up and securely store an API key for a provider of choice; make raw LLM API calls and structure messages correctly; parse responses and handle errors/rate limits	Industry Mentor
Module 3: GenAI & Agents	14	14.1	Prompt Engineering & Reasoning	Role prompting, zero-shot prompting, few-shot prompting, chain-of-thought prompting, prompt evaluation with small test sets, iterative prompt refinement, prompt failure analysis	Design effective zero-shot and few-shot prompts for specific tasks using role prompting; apply chain-of-thought reasoning to improve multi-step problem solving; evaluate and iteratively refine prompts using a defined test set and failure analysis	Industry Mentor
Module 3: GenAI & Agents	14	14.2	Master Class: Vectors & Linear Algebra The Mathematics of Space and Direction	Vectors from Scratch : What is a scalar vs a vector. Representing vectors as arrows and as coordinate lists. Vector addition, subtraction, scalar multiplication geometrically and algebraically. Magnitude using the Pythagorean theorem. Unit vectors. Dot Product & Angle Between Vectors : Formula: $a \cdot b = a b \cos(\theta)$. Computing it by hand. What the result tells you positive means the same direction, zero means perpendicular, negative means opposite. Cosine of an angle derived from the dot product formula step by step.	Add, subtract, and scale vectors both geometrically (arrows) and numerically Compute the magnitude of a 2D and 3D vector using Pythagoras Calculate the dot product of two vectors by hand Use the dot product formula to find the angle between two vectors Explain geometrically what it means for two vectors to be perpendicular (dot product = 0)	Professor
Module 3: GenAI & Agents	14	14.3	Structured Outputs with Pydantic	Pydantic BaseModel, field definitions, type validation, instructor library, patching OpenRouter client, extracting structured data from LLM responses, output validation, handling malformed outputs	Define structured output schemas using Pydantic BaseModel; patch an OpenRouter client with instructor to enforce structured JSON responses; validate and parse structured LLM outputs and handle malformed response gracefully	Industry Mentor
Module 3: GenAI & Agents	15	15.1	Getting Started with LangChain	LangChain setup, ChatOpenAI with OpenRouter base URL, PromptTemplate, ChatPromptTemplate, LLMChain, SequentialChain, why LangChain over raw API calls	Set up LangChain with OpenRouter as the LLM backend and explain why frameworks like LangChain exist; build prompt templates using PromptTemplate and ChatPromptTemplate; chain multiple LLM calls using LLMChain and SequentialChain	Industry Mentor
Module 3: GenAI & Agents	15	15.2	Building AI Workflows in LangChain	LangChain output parsers, PydanticOutputParser, JsonOutputParser, extraction chains, summarization chains, FAQ bot design, CSV copilot pattern, multi-step pipelines	Build extraction and summarization workflows using LangChain chains and output parsers; design a simple FAQ bot and CSV copilot using LangChain pipeline patterns; construct multi-step AI pipelines that pass structured outputs between steps	Industry Mentor

Module 3: GenAI & Agents	16	16.1	Master Class : Calculus Intuition - Change, Slope & Optimisation	The Idea of a Function & Rate of Change What is a function (input $\rightarrow$ output). Plotting simple functions ($y = x^2$, $y = x^3$). The slope of a straight line as rise/run. Average rate of change between two points. What happens as those two points get closer the intuition of a derivative without formal limits. Derivatives What They Mean Geometrically The derivative as the slope of a curve at a point. Tangent line vs secant line. Basic derivative rules: $d/dx(x^n) = nx^{n-1}$, constant rule, sum rule applied to polynomials only. Finding where a function is increasing, decreasing, or flat (derivative = 0). What a minimum and maximum look like. Optimisation Intuition — What does "minimise something" mean mathematically? Finding the minimum of a parabola $y = x^2$ by setting $dy/dx = 0$. The idea of "walking downhill" why the slope tells you which direction to move. Chain rule mentioned conceptually not computed.	Compute the average rate of change of a function between two points Explain what a derivative is using only the tangent line picture — no epsilon-delta Apply basic derivative rules to simple polynomials by hand (e.g., $f(x) = 3x^2 + 2x + 1$) Find the minimum or maximum of a simple function by setting its derivative to zero Explain in plain language why "setting the derivative to zero" finds the bottom of a curve	Professor
Module 3: GenAI & Agents	16	16.2	Text Embeddings & Semantic Search	sentence-transformers library, HuggingFaceEmbeddings, embedding generation, cosine similarity, dot product similarity, semantic vs keyword search, embedding visualization	Generate text embeddings locally using sentence-transformers without any paid API; compute cosine similarity between embeddings to find semantically related content; distinguish semantic search from keyword search and explain when each is appropriate	Industry Mentor
Module 3: GenAI & Agents	16	16.3	Chunking, Vector Stores & Retrieval	LangChain RecursiveCharacterTextSplitter, chunk size and overlap strategies, ChromaDB setup and indexing, FAISS setup and indexing, similarity search, MMR retrieval, VectorStoreRetriever, ChromaDB vs FAISS comparison	Chunk documents using RecursiveCharacterTextSplitter with appropriate size and overlap; index document chunks into both ChromaDB and FAISS and compare their trade-offs; retrieve relevant chunks using similarity search and MMR retrieval	Industry Mentor
Module 3: GenAI & Agents	17	17.1	Building a RAG Pipeline	LangChain RetrievalQA, create_retrieval_chain, combine_documents_chain, connecting retriever to LLM, prompt with context injection, grounded document Q&A, hallucination reduction, grounded vs ungrounded response comparison	Build an end-to-end RAG pipeline connecting a ChromaDB retriever to an LLM via LangChain; inject retrieved context into prompts to ground LLM responses in source documents; compare grounded vs ungrounded responses to analyze and reduce hallucination	Industry Mentor
Module 3: GenAI & Agents	17	17.2	Building an AI Agent with LangChain	LangChain Tool, create_tool, AgentExecutor, ReAct agent, initialize_agent, tool selection logic, multi-step action loops, max_iterations, handle_parsing_errors, mini-build: CSV Analyst or FAQ Helpdesk agent	Define custom tools and connect them to a LangChain AgentExecutor using the ReAct pattern; build a multi-step agent that selects and calls tools based on input; construct a working mini-agent build — either a CSV Analyst or FAQ Helpdesk — and control its execution safely	Industry Mentor
Module 3: GenAI & Agents	18	18.1	Agentic RAG, Evaluation & Guardrails	LangChain retriever as tool, create_retrieval_tool, agentic RAG pattern, RAGAS framework, answer correctness, context recall, faithfulness score, hallucination detection, guardrails design, privacy considerations, reliability principles, responsible AI framework	Wrap a ChromaDB retriever as a LangChain tool and integrate it into an AgentExecutor for agentic RAG; evaluate RAG pipeline quality using RAGAS metrics — answer correctness, context recall and faithfulness; implement guardrails and explain responsible AI principles around hallucination, privacy and reliability	Industry Mentor
Module 3: GenAI & Agents	18	18.2	Agent Orchestration with LangGraph	LangGraph StateGraph, nodes, edges, conditional edges, add_node, add_edge, compile(), state management across nodes, retry handling, multi-step agent flow design, LangGraph vs AgentExecutor, when to use each	Design a multi-step agent workflow using LangGraph StateGraph with defined nodes and transitions; implement conditional edges to handle branching logic and retry handling across agent steps; compare LangGraph orchestration with LangChain AgentExecutor and justify when to use each	Industry Mentor